

Functional Design Document

PDF Chatbot using Retrieval-Augmented Generation (RAG)

Project Name: PDF Chatbot

Date: June 27, 2026

Version: 1.0

Status: Active Development

1. Executive Summary

This document details the functional specifications and behavioral requirements of the PDF Chatbot system. It defines what the system does, how users interact with it, and the expected outputs for various inputs. The document serves as a bridge between technical architecture and user experience.

2. System Purpose & Scope

2.1 Purpose

The PDF Chatbot enables users to:

- Upload or reference PDF documents
- Ask natural language questions about the document content
- Receive accurate, context-aware answers sourced from the document
- Understand answer provenance through source references

2.2 Scope

Included:

- Single PDF document processing
- Natural language question answering
- Vector-based semantic search
- Real-time response generation
- Web-based user interface

****Out of Scope (Current Version):****

- Multiple document support
- Document upload functionality
- Chat history persistence
- Multi-user authentication
- Advanced analytics

3. User Profiles

3.1 Primary Users

****Knowledge Seekers****

- Goal: Extract information from documents quickly
- Skill Level: Non-technical
- Frequency: Ad-hoc usage
- Use Case: Research, learning, information discovery

****Business Analysts****

- Goal: Find specific information and trends
- Skill Level: Technical
- Frequency: Regular usage
- Use Case: Data analysis, reporting, insights

****Researchers****

- Goal: Comprehensive document understanding
- Skill Level: Technical
- Frequency: Extended sessions
- Use Case: Literature review, deep analysis

3.2 User Scenarios

****Scenario 1: Quick Lookup****

- User: "What is Kafka?"
- System: Returns brief definition with context
- Time: 2-5 seconds

****Scenario 2: Detailed Explanation****

- User: "Explain Kafka architecture and components"
- System: Provides comprehensive explanation

- Time: 5-10 seconds

****Scenario 3: Comparison Query****

- User: "What are the differences between producers and consumers in Kafka?"

- System: Returns comparative analysis

- Time: 3-8 seconds

****Scenario 4: Troubleshooting****

- User: "How do I fix replication lag in Kafka?"

- System: Provides relevant solutions and configurations

- Time: 2-7 seconds

4. Core Features

4.1 Feature List

4.2 Feature Descriptions

4.2.1 PDF Document Loading

****Functionality:****

- Load PDF file from specified path
- Extract text content from all pages
- Preserve document structure metadata
- Handle multi-page documents

****Input:****

- PDF file path: `G:\AI Project\pdf-chatbot\data\Kafka.pdf`

****Output:****

- List of document pages with content
- Metadata: number of pages, total content length

****Example:****

Input: "Kafka.pdf" (500 pages)

Output: 500 pages with text content extracted

4.2.2 Text Chunking

****Functionality:****

- Split documents into optimal chunks
- Maintain context with overlapping text
- Preserve paragraph boundaries when possible
- Generate chunk boundaries

****Parameters:****

- Chunk Size: 1000 characters
- Overlap: 200 characters

****Output:****

- List of text chunks with metadata

****Example:****

Input: 500-page document

Output: 450 chunks of ~1000 chars each with 200 char overlap

4.2.3 Vector Embedding

****Functionality:****

- Convert text chunks to numerical vectors
- Generate 384-dimensional embeddings
- Normalize vectors for comparison
- Store embeddings efficiently

****Model:****

- `sentence-transformers/all-MiniLM-L6-v2`
- Output: 384-dimensional vectors

****Example:****

Input: "What is Kafka?"

Output: [0.21, 0.44, 0.55, ..., 0.99] (384 dimensions)

4.2.4 Semantic Search

****Functionality:****

- Convert user question to embedding
- Calculate similarity to chunk embeddings
- Return top-k most relevant chunks
- Rank results by relevance

****Parameters:****

- k (number of results): 3

****Output:****

- Top 3 most relevant document chunks
- Similarity scores

****Example:****

Input: "What is Kafka?"

Output: [

{chunk: "Kafka is a distributed...", score: 0.87},

{chunk: "Apache Kafka is used for...", score: 0.85},

{chunk: "Kafka provides...", score: 0.82}

]

4.2.5 Answer Generation

****Functionality:****

- Combine retrieved documents and question into prompt
- Send to LLM for processing
- Generate natural language response
- Return complete answer to user

****Model:****

- Ollama llama3 (local LLM)
- Context window: Model-dependent

****Prompt Template:****

Context: [Retrieved document chunks]

Question: [User question]

Answer: [Generated response]

****Output:****

- Natural language answer
- Generation time

****Example:****

Input:

Context: "Kafka is a distributed event streaming platform..."

Question: "What is Kafka?"

Output: "Kafka is a distributed event streaming platform developed

4.2.6 Web User Interface

****Functionality:****

- Display application title and description

- Accept user questions via text input
- Trigger search and generation on button click
- Display loading indicator during processing
- Show results to user
- Maintain session state

****Components:****

- Page configuration (title, icon)
- Title heading with emoji
- Instructions text
- Question input field
- Ask button
- Loading spinner
- Success message
- Answer display

****Example Flow:****

User Interface

[Enter Question] -> [Ask Button] -> [Loading...] -> [Display Answer]

4.2.7 Answer Caching

****Functionality:****

- Store previously generated answers
- Return cached answer for repeated questions
- Reduce computation time
- Improve user experience

****Implementation:****

- Streamlit session state caching
- Decorator: `@st.cache_data``

****Example:****

First Query: "What is Kafka?"

- No cache hit
- Compute: ~5-10 seconds

Second Query: "What is Kafka?"

- Cache hit
- Return: ~0.1 seconds

4.2.8 Model Caching

****Functionality:****

- Load embedding model once per session
- Load FAISS database once per session
- Load LLM once per session
- Avoid redundant model initialization

****Implementation:****

- Streamlit resource caching
- Decorator: `@st.cache\_resource`

****Benefit:****

Session Start: Load models (~30 seconds)

First Query: Use cached models (~2 seconds)

Subsequent: Use cached models (~2 seconds each)

5. User Workflows

5.1 Main Workflow: Ask a Question

****Workflow Name:**** User Question-Answering

****Actors:****

- End User (asks questions)
- PDF Chatbot System (processes and responds)

****Preconditions:****

- System is running
- PDF document is loaded and indexed
- Models are loaded in memory

****Main Flow:****

1. User opens web application
2. User sees interface with input field
3. User types question in text input
4. User clicks "Ask" button
5. System validates question (non-empty)
6. System displays loading spinner
7. System retrieves relevant documents
8. System generates answer using LLM
9. System caches question and answer

10. System displays answer to user
11. User reads answer
12. User can ask follow-up question (loop to step 3)

****Postconditions:****

- User receives answer
- Answer is cached for future reference
- Session state is updated

****Alternate Flows:****

****A1: Empty Question****

- If user clicks Ask with empty question
- System does not process
- No loading spinner shown
- No answer displayed

****A2: Repeated Question****

- If user asks same question again
- System retrieves from cache
- Answer displayed immediately
- No LLM call needed

****A3: Network/System Error****

- If LLM is unavailable
- Error message displayed
- User prompted to retry

****Business Rules:****

- Only non-empty questions are processed
- Maximum question length: 5000 characters
- Answers generated within 10 second timeout
- Top 3 document chunks used for context

****Example:****

User: "What is Kafka?"

System Processing:

1. Convert question to embedding: [0.21, 0.44, ...]
2. Search FAISS: Find top 3 chunks
3. Build context: "Kafka is a distributed..."
4. Generate answer: "Kafka is a distributed event"
5. Display answer

User sees: "Answer: Kafka is a distributed event"

5.2 Data Processing Workflow: Ingest PDF

Workflow Name: Document Ingestion

Actors:

- Administrator (initiates)
- Ingestion System (processes)

Preconditions:

- PDF file exists at specified path
- Sufficient disk space for vectorstore
- HuggingFace models available

Main Flow:

1. Administrator runs `ingest.py`
2. System authenticates with HuggingFace
3. System loads PDF file
4. System extracts pages and content
5. System logs loading time and page count
6. System splits content into chunks
7. System logs chunk count and split time
8. System loads embedding model
9. System generates vectors for all chunks
10. System logs embedding time
11. System creates FAISS database
12. System saves database to disk
13. System logs success message and total time

Postconditions:

- FAISS vectorstore created and saved
- Document is ready for querying
- Performance metrics logged

Example:

Administrator: `python ingest.py`

System Output:

Loading PDF...time taken 2.45

Pages Loaded: 500

Chunks Created: 447

Splitting Documents time taken... 1.23
Generating embeddings...
Generating embeddings time taken... 8.90
Creating FAISS database...
Vector DB Saved Successfully
Creating FAISS database time taken... 3.45

6. Data Flow Specifications

6.1 Question Processing Data Flow

User Question
[Validation] --> Check non-empty
[Embedding Generation] --> Convert to vector (384-dim)
[FAISS Search] --> Similarity search (k=3)
[Document Retrieval] --> Extract top 3 chunks
[Context Building] --> Concatenate documents
[Prompt Construction] --> Question + Context
[LLM Generation] --> Call llama3 model
[Answer Output] --> Display result to user
[Caching] --> Store question + answer
User Response

6.2 Document Ingestion Data Flow

PDF File (Kafka.pdf)
[PyPDFLoader] --> Extract pages
Document Pages (500 pages)
[RecursiveCharacterTextSplitter]
Document Chunks (447 chunks)
[HuggingFaceEmbeddings]
Embedding Vectors (447 x 384)
[FAISS.from_documents]
FAISS Index
[db.save_local("vectorstore")]
Persisted Vector Store (index.faiss, index.pkl, docstore.pkl)

7. Business Rules & Constraints

7.1 Data Processing Rules

7.2 Performance Rules

7.3 Quality Rules

8. System Behavior Specifications

8.1 Input Specifications

8.1.1 User Question Input

****Input Type:**** Text String

****Format:****

- Plain text
- Natural language English
- May contain special characters
- May contain domain-specific terminology

****Validation Rules:****

- Non-empty (required)
- Length: 1-5000 characters
- No null values
- No binary content

****Valid Examples:****

"What is Kafka?"

"Explain the architecture of Kafka including producers and consumers"

"How do I configure Kafka for high throughput?"

"What are the differences between RabbitMQ and Kafka?"

****Invalid Examples:****

"" (empty)

"a" (too short is allowed, but empty not)

"\x00\x01\x02" (binary)

NULL (null value)

8.1.2 PDF Document Input

****Input Type:**** File

****Format:****

- PDF (.pdf extension)
- Binary PDF format
- Single file
- Any number of pages

****Validation Rules:****

- Must exist at specified path
- Must be readable
- Must be valid PDF structure
- Must contain extractable text

****Valid Location:****

G:\AI Project\pdf-chatbot\data\Kafka.pdf

8.2 Output Specifications

8.2.1 Answer Output

****Output Type:**** Text String

****Format:****

- Plain text
- Natural language English
- May span multiple paragraphs
- Generated by LLM

****Content Requirements:****

- Answers user's question directly
- Sourced from document content
- Factually accurate
- Complete and coherent

****Structure:****

[Direct answer to question]

[Supporting details]

[Context or examples]

[Related information if relevant]

****Example Output:****

"Kafka is a distributed event streaming platform developed by the Apache Software Foundation. It is designed to handle real-time data feeds with high throughput and low latency.

Kafka works by organizing data into topics, which are further divided into partitions for parallel processing. Producers send messages to topics, while consumers read messages from these topics.

The key benefits of Kafka include:

- High throughput for processing millions of messages per second
- Fault tolerance through replication
- Scalability through partitioning
- Durability through persistent storage"

8.2.2 Cache Storage

****Output Type:**** Session State Dictionary

****Format:****

```
{  
}
```

****Purpose:****

- Avoid recomputation
- Improve response time
- Track user session data

8.3 State Management

8.3.1 Session State Variables

8.3.2 State Transitions

Initial State

[User Opens App] -> Session created, answer = ""
[User Asks Question] -> answer = processing
[System Retrieves] -> answer = retrieved_docs
[LLM Generates] -> answer = generated_response
[Display Answer] -> answer shown to user
[User Asks Again] -> answer = new_response
[Cache Hit] -> answer = cached_response (instant)

9. Functional Requirements

9.1 FR: Document Processing

ID: FR-DOC-001

Title: PDF Document Loading

Description: System shall load PDF documents and extract text content

Requirements:

- Load file from specified path
- Extract all pages
- Preserve content order
- Handle multi-page documents
- Log loading statistics

Acceptance Criteria:

- Can load 500+ page documents
- Extracts 100% of text content
- Completes in <5 seconds
- Logs page count

Status: Implemented

ID: FR-DOC-002

Title: Text Chunking

Description: System shall split documents into contextual chunks

Requirements:

- Create chunks of 1000 characters
- Maintain 200 character overlap
- Preserve semantic meaning

- Generate unique identifiers

****Acceptance Criteria:****

- Creates chunks with specified size
- Overlap correctly applied
- Chunk count logged
- Process time tracked

****Status:**** Implemented

****ID:**** FR-DOC-003

****Title:**** Vector Embedding Generation

****Description:**** System shall convert text to embeddings

****Requirements:****

- Use all-MiniLM-L6-v2 model
- Generate 384-dimensional vectors
- Normalize embeddings
- Store efficiently

****Acceptance Criteria:****

- All chunks vectorized
- Embedding dimensions correct
- Process time logged
- FAISS DB created successfully

****Status:**** Implemented

9.2 FR: Question Answering

****ID:**** FR-QA-001

****Title:**** Semantic Document Search

****Description:**** System shall retrieve relevant documents using semantic search

****Requirements:****

- Convert question to embedding
- Search FAISS index
- Return top 3 results
- Rank by relevance score

****Acceptance Criteria:****

- Retrieves relevant documents
- Top 3 results returned
- Search completes in <100ms
- Results ranked properly

****Status:**** Implemented

****ID:**** FR-QA-002

****Title:**** Answer Generation

****Description:**** System shall generate natural language answers

****Requirements:****

- Use retrieved documents as context
- Call LLM for generation
- Generate complete answers
- Ensure factual accuracy

****Acceptance Criteria:****

- Answers generated from context
- No hallucinated content
- Completes in <10 seconds
- Grammar and coherence verified

****Status:**** Implemented

****ID:**** FR-QA-003

****Title:**** Question Validation

****Description:**** System shall validate user questions

****Requirements:****

- Reject empty questions
- Enforce character limits
- Sanitize input
- Display error messages

****Acceptance Criteria:****

- Empty questions rejected
- Max length enforced
- Clear error messages
- No system errors

****Status:**** Implemented (Basic)

9.3 FR: User Interface

ID: FR-UI-001

Title: Question Input

Description: System shall provide input field for user questions

Requirements:

- Display text input field
- Show placeholder text
- Support standard text input
- Clear on submit

Acceptance Criteria:

- Input field visible
- Accepts user text
- Placeholder text displays
- Keyboard input works

Status: Implemented

ID: FR-UI-002

Title: Answer Display

Description: System shall display answers to users

Requirements:

- Show answer text
- Format for readability
- Display success indicator
- Preserve formatting

Acceptance Criteria:

- Answer displays correctly
- Formatting preserved
- Success message shown
- Text is readable

Status: Implemented

ID: FR-UI-003

****Title:** Loading Indicator**

****Description:**** System shall show loading status during processing

****Requirements:****

- Display spinner during processing
- Show progress message
- Hide when complete
- Clear user feedback

****Acceptance Criteria:****

- Spinner appears during processing
- Message is clear
- Spinner disappears on completion
- No visual glitches

****Status:**** Implemented

9.4 FR: Performance

****ID:**** FR-PERF-001

****Title:**** Response Caching

****Description:**** System shall cache responses to repeated queries

****Requirements:****

- Cache question-answer pairs
- Return cached results instantly
- Reduce computation

****Acceptance Criteria:****

- Cached queries return in <1 second
- Cache hit rate tracked
- No stale data returned

****Status:**** Implemented

****ID:**** FR-PERF-002

****Title:**** Model Caching

****Description:**** System shall cache loaded models for session

****Requirements:****

- Load models once per session

- Reuse for all queries
- Reduce startup time

****Acceptance Criteria:****

- Models load once
- All queries use cached models
- Session persistence verified

****Status:**** Implemented

10. Non-Functional Requirements

10.1 Performance Requirements

10.2 Reliability Requirements

10.3 Usability Requirements

10.4 Security Requirements

10.5 Scalability Requirements

11. Error Handling

11.1 Error Scenarios

11.1.1 PDF Loading Errors

****Scenario:**** PDF file not found

****Trigger:****

PyPDFLoader("invalid_path.pdf")

****Current Behavior:****

- Runtime error
- Application crashes
- User sees error stack trace

****Expected Behavior:****

- Graceful error message
- User-friendly error display
- Recovery option

****Resolution:****

- Add file existence check
- Display error modal
- Suggest corrective action

11.1.2 Model Loading Errors

****Scenario:**** HuggingFace models unavailable

****Trigger:****

- Network connectivity issue
- Model download failure
- Disk space insufficient

****Current Behavior:****

- Download error
- Application hangs
- User frustrated

****Expected Behavior:****

- Timeout after 30 seconds
- Error message displayed
- Retry option provided

****Resolution:****

- Implement timeout
- Add retry logic
- Cache offline models

11.1.3 LLM Generation Errors

****Scenario:**** Ollama server unavailable

****Trigger:****

- Ollama not running
- Network connectivity issue
- Model not loaded

****Current Behavior:****

- Connection error
- Blank response
- No error message

****Expected Behavior:****

- Clear error message
- Suggestion to restart Ollama
- Retry option

****Resolution:****

- Add connection check
- Display helpful message
- Implement retry mechanism

11.1.4 FAISS Index Errors

****Scenario:**** Corrupted vector store

****Trigger:****

- Incomplete vectorstore save
- File corruption
- Version mismatch

****Current Behavior:****

- Deserialization error
- Application crash

****Expected Behavior:****

- Error notification
- Regeneration option

****Resolution:****

- Add index validation
- Implement fallback mechanism
- Provide regeneration option

11.2 Error Handling Strategy

12. Test Cases

12.1 Functional Test Cases

****TC-QA-001: Simple Question****

****TC-QA-002: Complex Question****

****TC-QA-003: Repeated Question (Cache)****

****TC-UI-001: Empty Question****

****TC-PERF-001: Model Loading****

****TC-PERF-002: Query Response****

13. Integration Points

13.1 External Systems

13.2 Data Interfaces

****PDF Input:****

File: Kafka.pdf (local filesystem)

Format: Binary PDF

Loader: PyPDFLoader

****Vector Store:****

Location: vectorstore/ directory

Format: FAISS index + metadata

Access: FAISS.load_local()

****Model Cache:****

Location: .cache/ directory

Format: PyTorch/Hugging Face format

Access: HuggingFaceEmbeddings()

14. Assumptions & Constraints

14.1 Assumptions

14.2 Constraints

15. Future Functional Enhancements

15.1 Short Term (1-2 months)

****FE-001: Multi-Document Support****

- Load multiple PDFs
- Cross-document search
- Document selection UI
- Combined indexing

****FE-002: Source Citations****

- Track answer sources
- Display page numbers
- Highlight source text

- Citation formatting

****FE-003: Chat History****

- Store conversation history
- Previous query browsing
- Context continuity
- Export conversations

****FE-004: Error Recovery****

- Graceful error handling
- User-friendly messages
- Automatic retries
- Recovery suggestions

15.2 Medium Term (2-4 months)

****FE-005: Document Upload****

- Allow users to upload PDFs
- Processing queue
- Status tracking
- Upload validation

****FE-006: Advanced Search****

- Hybrid search (semantic + keyword)
- Filters and facets
- Query expansion
- Result ranking

****FE-007: Answer Refinement****

- Follow-up questions
- Answer clarification
- Related topics
- Confidence scoring

****FE-008: User Preferences****

- Answer length control
- Language selection
- Detail level preference
- Display themes

15.3 Long Term (4+ months)

****FE-009: Analytics Dashboard****

- Query analytics
- Popular questions
- Usage patterns
- Performance metrics

****FE-010: Collaborative Features****

- User authentication
- Multi-user support
- Question sharing
- Comments/annotations

****FE-011: Advanced RAG****

- Reranking models
- Query expansion
- Hybrid retrieval
- Multi-hop reasoning

****FE-012: Mobile App****

- Mobile UI
- Responsive design
- Offline support
- Native app option

16. Success Metrics

16.1 Functional Success Metrics

16.2 Performance Success Metrics

16.3 User Experience Metrics

17. Conclusion

This Functional Design Document provides a comprehensive specification of the PDF Chatbot system's capabilities, behavior, and requirements. It serves as the blueprint for development, testing, and user acceptance validation.

The system successfully implements a RAG-based question-answering platform that provides users with an intuitive interface for querying PDF documents. Key features include semantic search, LLM-based answer generation, and performance optimizations through caching.

Future enhancements will expand functionality to support multiple documents, advanced search capabilities, and enterprise features, positioning the system for broader adoption and use cases.

Appendix A: Glossary

Appendix B: References

Document Version: 1.0
Last Updated: June 27, 2026
Next Review: September 27, 2026